

Admin Dashboard & Role-Based Access Control

Admin Dashboard & Role-Based Access Control

Pemrograman Berbasis Kerangka Kerja (PBKK) - Pertemuan 10

Departemen Teknik Informatika, FTEIC, Institut Teknologi Sepuluh Nopember

Agenda Pembelajaran & Target Hari Ini

1. Integrasi Template Admin — Membangun antarmuka administrator professional menggunakan AdminLTE, Tabler, atau Filament.

2. Role & Permission Management — Mengontrol hak akses pengguna secara dinamis berbasis Spatie Role & Permission.

3. Studi Kasus Dashboard Admin — Mengintegrasikan sistem autentikasi multi-user dengan widget ringkasan statistik database.

4. Pencarian & Filter Data — Mengembangkan modul kueri pencarian, filter, dan pagination dinamis menggunakan kueri when().

5. Contoh Soal & Evaluasi Koding — Diskusi dan simulasi 5 contoh soal program siap industri.

6. Penugasan Kelompok Akhir Semester — Kickoff rancangan dan persiapan proyek platform Agentic AI ITS.

BAGIAN 1

Integrasi Template Admin Dashboard

Kebutuhan Dashboard Khusus untuk Administrator

• Pemisahan UI Publik vs Internal — Pengguna biasa dan administrator membutuhkan antarmuka visual yang berbeda secara fungsionalitas.

• Pusat Pengelolaan Data (Backoffice) — Admin dashboard bertindak sebagai panel pusat pemantauan statistik, log aktivitas, dan operasional CRUD data sensitif.

• Standardisasi Navigasi — Menyediakan Sidebar dinamis, Topbar, Panel Widget Info, dan layouting tabel terstruktur secara konsisten.

• Efisiensi Pengembangan — Memanfaatkan template siap pakai menghemat ratusan jam pengerjaan kustomisasi CSS/JS dari nol.

AdminLTE: Kerangka Dashboard Klasik & Stabil

• Berbasis Bootstrap 4 & jQuery — Template terpopuler di ekosistem web yang terkenal stabil, responsif, dan kaya fitur.

• Ratusan Komponen Visual Bawaan — Menyediakan chart (ChartJS), ikon, box widget statistik, formulir modern, dan pagination siap pakai.

• Kompatibilitas Tinggi — Mudah diintegrasikan dengan Laravel melalui paket kustom yang dikelola secara aktif oleh komunitas.

• Keandalan Industri — Teruji di berbagai sistem backoffice perusahaan karena fungsionalitas tata letak sidebar dan header yang solid.

Langkah Integrasi AdminLTE pada Proyek Laravel

- Instalasi Paket jeroennoten/laravel-adminlte via Composer:

  $ composer require jeroennoten/laravel-adminlte

- Mempublikasikan Berkas Konfigurasi dan Aset:

  $ php artisan adminlte:install

- Paket ini secara otomatis menyediakan file konfigurasi terpusat di config/adminlte.php untuk mengatur seluruh perilaku dashboard.

Konfigurasi Sidebar dan Master Layout Blade

• Menu Sidebar Dinamis — Menu, sub-menu, ikon, dan filter otorisasi didaftarkan langsung pada array di file config/adminlte.php.

• Pewarisan Layout yang Super Cepat — Halaman view admin cukup mewarisi layout bawaan tanpa membuat file kerangka header-sidebar sendiri:

```
@extends('adminlte::page')

@section('title', 'Dashboard Admin')

@section('content_header') <h1>Dashboard</h1> @endsection

@section('content') <p>Selamat datang di sistem!</p> @endsection
```

Tabler: Alternatif Desain Dashboard Modern & Ringan

- Berbasis Bootstrap 5 murni tanpa ketergantungan jQuery (modern, ringan, dan cepat).
- Estetika Desain Minimalis — Memiliki palet warna bersih, sudut melengkung modern, dan set ikon yang ramah mata (Tabler Icons).
- Integrasi Laravel Tabler — Paket komunitas mempermudah instalasi dan scaffolding layout Blade secara modular.
- Cocok untuk Aplikasi Kontemporer — Sangat ideal jika Anda membangun dashboard modern yang membutuhkan kinerja tinggi dan rendering cepat.

Filament PHP: Era Baru Admin Generator TALL Stack

• Generasi Baru Dashboard Laravel — Memanfaatkan kekuatan TALL Stack (Tailwind CSS, Alpine.js, Laravel, dan Livewire).

• Interaktivitas Reaktif Tanpa JavaScript Manual — Semua manipulasi UI dilakukan reaktif di sisi server berkat Livewire.

• Deklaratif CRUD Generator — Mengurangi kodingan file views HTML hingga 90% karena tabel dan form dibuat dari deklarasi PHP murni.

• Ekosistem Sangat Kaya — Terintegrasi dengan form builder, table builder, widgets, dan autentikasi bawaan dalam sekali instal.

Cara Kerja Pemodelan Filament Resource

• Membuat Resource CRUD Instan untuk Model Produk:

$ php artisan make:filament-resource Produk

• Otomatisasi Berkas — Perintah di atas akan memproduksi form schema dan skema kolom tabel secara instan di dalam file ProdukResource.php.

• Kemudahan Modifikasi — Pengembang cukup merinci skema form (misal: TextInput, Select) dan kolom tabel (misal: TextColumn, ImageColumn) langsung di file tersebut.

BAGIAN 2

Role & Permission Management

Konsep RBAC: Membedakan Antara Role dan Permission

• Role (Peran Pengguna) — Mewakili posisi atau jabatan dalam sistem organisasi. Contoh: Admin, Dosen, Asisten Dosen, Mahasiswa.

• Permission (Izin Aksi) — Hak akses spesifik untuk melakukan tindakan operasional tertentu pada data. Contoh: 'tambah-nilai', 'hapus-tugas'.

• Hubungan Relasi — Satu Peran (Role) dapat memiliki banyak Izin (Permissions). Satu Pengguna (User) dapat ditautkan dengan satu atau lebih Peran.

• Fleksibilitas Keamanan — Memisahkan hak akses menjadi unit-unit izin kecil memudahkan kustomisasi izin tanpa membongkar logika rute atau kontroler.

Spatie Laravel Permission: Standar Industri Global

• Paket Otorisasi Terpopuler — Menawarkan manajemen peran (Roles) dan izin (Permissions) dinamis yang disimpan langsung ke database relasional.

• Fleksibilitas Tinggi — Aturan izin dapat diberikan langsung kepada Peran (Role) atau secara spesifik langsung ke Pengguna (User) tertentu.

• Keandalan Teruji — Didukung oleh caching bawaan Laravel secara otomatis untuk meminimalkan beban kueri database pada setiap request halaman.

• Integrasi Blade & Controller — Menyediakan rantaian metode intuitif, middleware, dan tag direktif Blade bawaan yang sangat bersih.

Instalasi dan Migrasi Paket Spatie Permission

- Unduh paket Spatie melalui Composer terminal:

  $ composer require spatie/laravel-permission

- Publikasikan berkas migrasi database fisik:

  $ php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider"

- Jalankan migrasi untuk membuat 5 tabel otorisasi relasional:

  $ php artisan migrate

Mengaktifkan Trait HasRoles pada Model User

- Mengimpor Trait HasRoles di baris atas model User:

  use Spatie\Permission\Traits\HasRoles;

- Menyisipkan Trait ke dalam deklarasi kelas Model User:

  class User extends Authenticatable {

      use HasRoles;

  }

- Trait ini secara otomatis membekali model User dengan puluhan asisten metode manipulasi peran dan izin.

Membuat Role dan Permission Secara Programmatis

- Menggunakan Model Spatie untuk Inisialisasi Baru:

  use Spatie\Permission\Models\Role;

  use Spatie\Permission\Models\Permission;

- Inisialisasi Izin Baru (Permissions):

  Permission::create(['name' => 'edit-tugas']);

- Inisialisasi Peran Baru (Role) dan Pemberian Izin:

  $roleDosen = Role::create(['name' => 'dosen']);

  $roleDosen->givePermissionTo('edit-tugas');

Menghubungkan Akun Pengguna dengan Peran

- Memberikan Peran Langsung pada Objek User:

  $user = User::find(1);

  $user->assignRole('dosen');
- Menghapus Peran dari Objek User:

  $user->removeRole('dosen');
- Melakukan Sinkronisasi Peran (Mengganti Peran Lama dengan Peran Baru):

  $user->syncRoles(['asisten-dosen']);
- Pemeriksaan Boolean Hak Akses: $user->hasRole('admin') atau $user->can('edit-tugas')

Melindungi Rute Menggunakan Middleware Spatie

- Mengamankan rute dari akses pengguna tak berhak menggunakan filter middleware Spatie bawaan:

  Route::get('/admin/panel', [AdminController::class, 'index'])->middleware('role:admin');
- Melindungi rute spesifik berdasarkan Izin (Permission) eksklusif:

  Route::delete('/tugas/{id}', [TugasController::class, 'destroy'])->middleware('permission:hapus-tugas');
- Menyusun multi-hak akses menggunakan operator OR: 'role:admin|dosen'

Proteksi Elemen Visual UI Menggunakan Blade Directive

- Menyembunyikan tombol 'Hapus' atau aksi transaksional sensitif dari layar pengguna biasa:

```
@can('hapus-tugas')
    <button class="btn btn-danger">Hapus Tugas</button>
@endcan
```

- Melakukan filter visual berdasarkan Peran (Role) spesifik secara instan:

```
@role('dosen')
    <div class="alert alert-info">Menu Khusus Dosen Wali</div>
@endrole
```

BAGIAN 3

Studi Kasus: Dashboard Admin Terintegrasi

Integrasi Dashboard Admin dengan Autentikasi Multi-User

• Skenario Kasus — Menyatukan modul CRUD Dosen & Mahasiswa dari pertemuan sebelumnya ke dalam kerangka master layout AdminLTE.

• Pemisahan Kontrol Akses — Halaman login dipusatkan lewat Laravel Breeze. Pengguna biasa akan dialihkan ke beranda publik.

• Deteksi Pengalihan — Saat admin login sukses, sistem mendeteksi peran Spatie dan melakukan redirect otomatis ke panel khusus `/admin/dashboard`.

• Keamanan Berlapis — Memastikan tidak ada pengguna non-admin yang dapat membypass URL dashboard admin meskipun mengetik manual di address bar.

Alur Pengalihan Rute Login Admin vs Publik

• Modifikasi LoginResponse — Mengubah rute fallback pasca login sukses pada file RouteServiceProvider atau Middleware Authenticate bawaan.

• Logika Pengalihan Dinamis:

```
if ($user->hasRole('admin')) {
    return redirect()->route('admin.dashboard');
}
return redirect()->route('home');
```

• Penguncian Grup Rute Admin — Mengelompokkan seluruh rute panel admin di bawah prefix `/admin` dan middleware `auth` serta `role:admin`.

Visual Widget Info Dashboard Dinamis

Total Mahasiswa

- Sumber Data: Model Mahasiswa
- Kueri: Mahasiswa::count()
- Menampilkan kuantitas seluruh mahasiswa aktif yang terdaftar di database.
- Terupdate real-time.

Total Staf Dosen

- Sumber Data: Model Dosen
- Kueri: Dosen::count()
- Menampilkan kuantitas dosen wali FTEIC ITS.
- Berguna untuk pemantauan rasio.

Total Tugas Aktif

- Sumber Data: Model Tugas
- Kueri: Tugas::count()
- Menampilkan jumlah tugas kuliah aktif yang belum melewati tenggat waktu.

Keamanan Otorisasi Berlapis di Sisi Server

• Aturan Utama Keamanan — Proteksi visual (menyembunyikan tombol via @can) hanyalah bagian dari kenyamanan antarmuka pengguna (UX).

• Ancaman API/Bypass Request — Pengguna jahat dapat menembak endpoint URL request (POST/DELETE) secara manual lewat aplikasi seperti Postman.

• Solusi Pengamanan Ganda — Mengunci logika eksekusi di Controller menggunakan otorisasi server-side murni:

```
$this->authorize('hapus-tugas', $tugas);
```

atau menggunakan middleware rute di routes/web.php secara ketat.

BAGIAN 4

Pencarian & Filter Data Tingkat Lanjut

Kelemahan Penulisan Kueri Filter Konvensional

• Kodingan yang Gemuk dan Kotor — Menulis struktur pengondisian if-else bertingkat di dalam Controller merusak kerapian dan keterbacaan kode.

• Pola Spaghetti Kueri — Penggabungan query builder yang tidak rapi sering kali memicu kesalahan sintaks SQL saat parameter filter kosong.

• Kurang Scalable — Menambahkan parameter pencarian baru (misal: filter departemen) menuntut penulisan blok pengondisian baru yang berulang.

• Solusi Laravel — Memanfaatkan fungsionalitas penengah kueri kondisional `when()` untuk menyaring data secara elegan.

Query Kondisional Bersih Menggunakan Method when()

• Sintaks Elegan when() — Menyisipkan klausa kueri database (seperti WHERE) hanya apabila variabel penilai bernilai TRUE:

```
$mahasiswa = Mahasiswa::query()
    ->when($request->search, function ($query, $search) {
        return $query->where('nama', 'like', "%$search%");
    })->get();
```

• Tanpa Blok If Manual — Jika `$request->search` bernilai null, Laravel otomatis mengabaikan callback fungsi dan melanjutkan eksekusi query dasar.

Mengunci Parameter Filter pada Link Pagination

• Masalah Reset Filter — Saat pengguna berpindah halaman pagination (klik halaman 2), data filter pencarian sering kali hilang karena URL ter-reset.

• Solusi: withQueryString() — Menginstruksikan Laravel untuk melampirkan seluruh parameter input aktif ke dalam link halaman pagination secara dinamis:

```
$mahasiswa = Mahasiswa::where('aktif', true)
    ->paginate(10)
    ->withQueryString();
```

• Hasilnya, filter pencarian akan tetap aktif terjaga di URL meskipun pengguna menjelajahi halaman tabel terdalam.

Rancangan Filter Multi-Variabel di Tabel Admin

🔍 Keyword Search

- Fungsi: Mencari teks bebas
- Target Kolom: 'nama' / 'nrp'
- Melakukan pencarian menggunakan klausa LIKE dinamis melalui input tunggal form filter.

🏢 Departemen Filter

- Fungsi: Dropdown penyaring
- Target Kolom: 'departemen_id'
- Menyaring daftar mahasiswa berdasarkan jurusan spesifik di fakultas FTEIC.

☐ Status Filter

- Fungsi: Tombol toggle status
- Target Kolom: 'is_active'
- Memisahkan tampilan data antara mahasiswa aktif dan mahasiswa yang sedang cuti akademik.

Optimasi Kinerja Kueri Pencarian Skala Besar

• Pentingnya Indeks Basis Data — Selalu pasang indeks fisik (database index) pada kolom tabel yang sering dicari (seperti `nrp`, `nama`, `nip`) lewat file migration.

• Hindari Klausa Wildcard Kiri — Pencarian LIKE '%keyword' memaksa basis data melakukan scanning seluruh tabel fisik (*full-table scan*) yang sangat lambat.

• Proteksi Injeksi SQL — Query Builder Laravel otomatis menggunakan PDO parameter binding di latar belakang untuk mensterilkan seluruh input dinamis.

• Eager Loading Wajib — Pastikan kueri pencarian relasi tetap dibekali method `with()` untuk mencegah degradasi kinerja akibat N+1 query.

BAGIAN 5

Contoh Soal & Solusi Kodingan

Contoh 1: Registrasi Peran & Izin di Seeder

Deskripsi & Analisis Soal

Skenario Kasus:

Dosen meminta Anda untuk menginisialisasi sistem otorisasi baru di file seeder agar sistem siap pakai saat migrasi.

Tugas:

Buat berkas RoleAndPermissionSeeder untuk mendaftarkan peran 'asisten-dosen' dan menautkannya dengan izin 'kelola-nilai'.

💻 IMPLEMENTASI KODE

```
use Illuminate\Database\Seeder;
use Spatie\Permission\Models\Role;
use Spatie\Permission\Models\Permission;

class RoleAndPermissionSeeder extends Seeder {
    public function run(): void {
        // Membuat Izin Baru
        $perm = Permission::firstOrCreate([
            'name' => 'kelola-nilai'
        ]);

        // Membuat Peran Baru
        $role = Role::firstOrCreate([
            'name' => 'asisten-dosen'
        ]);

        // Menautkan Peran dengan Izin
        $role->givePermissionTo($perm);
    }
}
```

Contoh 2: Proteksi Aksi Controller Berizin

Deskripsi & Analisis Soal

Skenario Kasus:

Anda harus memastikan bahwa aksi penghapusan data tugas di TugasController tidak bisa ditembus oleh user ilegal.

Tugas:

Gunakan middleware Spatie di dalam method rute atau controller untuk memproteksi endpoint destroy.

💻 IMPLEMENTASI KODE

```
use App\Http\Controllers\Controller;
use App\Models\Tugas;
use Illuminate\Http\Request;

class TugasController extends Controller {
    public function __construct() {
        // Mengamankan method destroy dengan middleware
        $this->middleware('permission:hapus-tugas')
             ->only('destroy');
    }

    public function destroy(Tugas $tugas) {
        $tugas->delete();
        return redirect()->back()
            ->with('success', 'Tugas berhasil didelete.');
    }
}
```

Contoh 3: Kueri Pencarian Multi-Kolom when()

Deskripsi & Analisis Soal

Skenario Kasus:

Form filter pencarian di dashboard admin harus mampu melayani pencarian mahasiswa berdasarkan kecocokan nama ATAU NRP sekaligus.

Tugas:

Susun kueri Eloquent query menggunakan sub-query closure di dalam metode when().

💻 IMPLEMENTASI KODE

```
public function index(Request $request) {
    $search = $request->input('search');

    $mahasiswas = Mahasiswa::query()
        ->when($search, function ($query, $search) {
            return $query->where(function ($q) use ($search) {
                $q->where('nama', 'like', "%$search%")
                  ->orWhere('nrp', 'like', "%$search%");
            });
        })
        ->paginate(10);

    return view('admin.mahasiswa.index', compact('mahasiswas'));
}
```

Contoh 4: Filter Bertingkat & Pagination Aman

Deskripsi & Analisis Soal

Skenario Kasus:

Halaman kelola tugas memiliki filter dropdown departemen dan search box. Paginasi tabel harus tetap membawa filter tersebut.

Tugas:

Membangun kueri dinamis multi-filter dan mengunci parameter link menggunakan withQueryString().

💻 IMPLEMENTASI KODE

```
public function listTugas(Request $request) {
    $tugas = Tugas::query()
        ->when($request->search, function ($q, $search) {
            return $q->where('judul', 'like', "%$search%");
        })
        ->when($request->departemen_id, function ($q, $deptId) {
            return $q->where('departemen_id', $deptId);
        })
        ->paginate(15)
        ->withQueryString(); // Kunci query parameter

    return view('admin.tugas.index', compact('tugas'));
}
```

Contoh 5: Pengamanan Rute Group Multi-Role

Deskripsi & Analisis Soal

Skenario Kasus:

Aplikasi Anda memiliki rute dashboard akademik khusus admin dan dosen yang tidak boleh diakses oleh mahasiswa.

Tugas:

Tuliskan konfigurasi Route Group di routes/web.php menggunakan middleware peran gabungan Spatie.

💻 IMPLEMENTASI KODE

```
use App\Http\Controllers\Admin\DashboardController;
use Illuminate\Support\Facades\Route;

// Mengelompokkan rute di bawah pengamanan ganda
Route::middleware(['auth', 'role:admin|dosen'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {

        Route::get('/dashboard', [DashboardController::class,
'index'])
            ->name('dashboard');

        // Tambahan rute kelola data internal lainnya...
    });
```

BAGIAN 6

Tugas Mandiri & Tantangan Mingguan

Tugas Mandiri: Dashboard & RBAC Implementation

• Tujuan Tugas — Mahasiswa wajib mengembangkan kelanjutan proyek portal akademik mandiri dengan mengintegrasikan kerangka visual panel admin professional dan memasang pengaman otorisasi dinamis.

• Integrasi UI Dashboard — Memanfaatkan paket AdminLTE atau menginisialisasi Filament panel admin pada proyek Laravel masing-masing.

• Konfigurasi Peran (Roles) — Menerapkan paket Spatie Laravel Permission dan melahirkan minimal dua peran utama: 'Admin' (Akses Penuh) dan 'Staf' (Akses Terbatas).

• Hak Eksklusif — Menu tertentu seperti 'Hapus Data' hanya boleh dirender visual di Blade dan dieksekusi di Controller apabila pengguna memegang peran Admin murni.

Spesifikasi Teknis Tugas (Aturan Wajib)

1. Integrasi UI Backoffice — Wajib menggunakan AdminLTE layouting (mewarisi @extends('adminlte::page')) atau Filament panel dashboard murni.

2. Sistem Autentikasi — Dashboard harus berada di bawah pengaman login terverifikasi via Laravel Breeze.

3. Otorisasi DB Spatie — Hak akses user disimpan dinamis ke database fisik menggunakan tabel otorisasi paket Spatie.

4. Widget Dashboard Agregat — Menampilkan minimal 3 widget box statistik dinamis di beranda admin (total data dari kueri model count()).

5. Pengaman Controller — Wajib melampirkan constructor middleware `permission:` atau otorisasi manual server-side demi mencegah kebocoran URL.

Tantangan Tambahan / Challenge (Nilai Plus A+)

• Filter Pencarian dan Kueri when() — Mengembangkan form pencarian data di halaman tabel admin menggunakan query kondisional `when()` agar kodingan Controller tetap bersih.

• Pengunci Link Pagination — Mencegah hilangnya filter kata kunci saat berpindah halaman tabel data menggunakan method `withQueryString()`.

• Auto-Seeder Faker Realistis — Menyediakan file database seeder otomatis yang langsung menyiram puluhan data dummy real-time menggunakan Faker generator saat proses instalasi awal.

• Auto-Login Test Accounts — Menyediakan tombol login instan kustom pada halaman login awal untuk mempermudah proses evaluasi demo dosen/asdos di kelas.

Kriteria Evaluasi & Rubrik Penilaian

• Integrasi UI Dashboard & Widgets (Bobot 30%) — Penataan master layout, kerapian elemen widget statistik, dan kebersihan visual.

• Keamanan Otorisasi Spatie RBAC (Bobot 30%) — Keberhasilan penguncian rute, perlindungan tombol visual Blade, dan integrasi database roles.

• Modul Pencarian & Filter when() (Bobot 20%) — Keefektifan fungsionalitas pencarian data tanpa merusak parameter pagination.

• Kerapian Kode Standar PSR-12 (Bobot 10%) — Standar kerapian penulisan script PHP, penggunaan namespace, dan penataan indentation.

• Keterampilan Demo & Presentasi (Bobot 10%) — Kelancaran melakukan demo live multi-user (login admin vs login staf) di depan kelas.

Ketentuan Pengumpulan & Sesi Demo Minggu Depan

• Batas Waktu Pengumpulan — Berkas link repositori GitHub wajib diunggah ke portal LMS ITS paling lambat H-1 sebelum Pertemuan 11 dimulai.

• Kelengkapan README — File `README.md` repositori wajib menyertakan dokumentasi instalasi proyek, detail akun uji coba (Auto-Login Credentials), serta tangkapan layar (screenshot) beranda dashboard.

• Konfigurasi Git Aman — Memastikan berkas `.env` dan folder `vendor/` tidak masuk ke repositori GitHub (memanfaatkan berkas pengaman `.gitignore` bawaan).

• Sesi Live Demo Acak — Pada awal Pertemuan 11, dosen akan menunjuk beberapa mahasiswa secara acak untuk melakukan demo kode langsung (maksimal 3 menit).